

ISLAMIC UNIVERSITY OF TECHNOLOGY



SWE 4301

LECTURE 14

Topic: Software Design & Design Smells

November 23, 2024

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 What is software design?	3
1.1 What Goes Wrong with Software?	5
2 Design SmellsThe Odors of Rotting Software	6
2.1 What Stimulates the Software to Rot?	8
2.2 HOW TO ADDRESS SMELLS?	8
2.3 DESIGN PRINCIPLES-BASED CLASSIFICATION OF SMELLS	9

1 WHAT IS SOFTWARE DESIGN?

Reference: PPP chapter 7 (What's agile design)

In 1992, Jack Reeves wrote a seminal article in the C++ Journal entitled What is Software Design? Reeves argues that the design of a software system is documented primarily by its source code. The diagrams representing the source code are supportive of the design and are not the design itself.

We will often talk about The Design. You should not take that to mean a set of UML diagrams separate from the code. A set of UML diagrams may represent parts of a design, but it is not the design. The design of a software project is an abstract concept. It has to do with the overall shape and structure of the program as well as the detailed shape and structure of each module, class, and method. It can be represented by many different media, but its final embodiment is source code. In the end, the source code is the design.

The final goal of any engineering activity is the some type of documentation. When a design effort is complete, the design documentation is turned over to the manufacturing team. This is a completely different group with completely different skills from the design team. If the design documents truly represent a complete design, the manufacturing team can proceed to build the product. In fact, they can proceed to build lots of the product, all without any further intervention of the designers.

After reviewing the software development life cycle as I understood it, I concluded that the only software documentation that actually seems to satisfy the criteria of an engineering design is the source code listings.

This article assumes that final source code is the real software design and then examines some of the consequences of that assumption. There is one consequence of considering code as software design that completely overwhelms all others. It is so important and so obvious that it is a total blind spot for most software organizations. This is the fact that software is cheap to build. It does not qualify as inexpensive; it is so cheap it is almost free. If source code is a software design, then actually building software is done by compilers and linkers. We often refer to the process of compiling and linking a complete software system as "doing a build". The capital investment in software construction equipment is low all it really takes is a computer, an editor, a compiler, and a linker. Once a build environment is available, then actually doing a software build just takes a little time.

Another consequence of considering source code as software design is the fact that a software design is relatively easy to create, at least in the mechanical sense. Writing (i.e.,

designing) a typical software module of 50 to 100 lines of code is usually only a couple of day's effort. It is tempting to ask if there is any other engineering discipline that can produce designs of such complexity as software in such a short time, but first we have to figure out how to measure and compare complexity. Nevertheless, it is obvious that software designs get very large rather quickly.

Given that software designs are relatively easy to turn out, and essentially free to build, an unsurprising revelation is that software designs tend to be incredibly large and complex. This may seem obvious but the magnitude of the problem is often ignored. School projects often end up being several thousand lines of code. There are software products with 10,000 line designs that are given away by their designers. We have long since passed the point where simple software is of much interest. Typical commercial software products have designs that consist of hundreds of thousands of lines. Additionally, software designs are almost always constantly evolving. While the current design may only be a few thousand lines of code, many times that may actually have been written over the life of the product.

While there are certainly examples of hardware designs that are arguably as complex as software designs, note two facts about modern hardware. One, complex hardware engineering efforts are not always as free of bugs as software critics would have us believe. Major microprocessors have been shipped with errors in their logic, bridges collapsed, dams broken, airliners fallen out of the sky, and thousands of automobiles and other consumer products have been recalled - all within recent memory and all the result of design errors. Second, complex hardware designs have correspondingly complex and expensive build phases. As a result, the ability to manufacture such systems limits the number of companies that produce truly complex hardware designs. No such limitations exist for software.

Designing software is an exercise in managing complexity. The complexity exists within the software design itself, within the software organization of the company, and within the industry as a whole. Software design is very similar to systems design. It can span multiple technologies and often involves multiple sub-disciplines. Software specifications tend to be fluid, and change rapidly and often, usually while the design process is still going on. Software development teams also tend to be fluid, likewise often changing in the middle of the design process. In many ways, software bears more resemblance to complex social or organic systems than to hardware.

The general consensus is that when real engineers get through with a design, no matter how complex, they are pretty sure it will work. They are also pretty sure it can be built using accepted construction techniques. In order for this to happen, hardware engineers spend a considerable amount of time validating and refining their designs. Consider a bridge design,

for example. Before such a design is actually built, the engineers do structural analysis; they build computer models and run simulations; they build scale models and test them in wind tunnels or other ways. In short, the designers do everything they could think of to make sure the design is a good design before it is built. The design of the new airliner is even worse; for those, full-scale prototypes must be built and tested to validate the design predictions.

It seems obvious to most people that software designs do not go through the same rigorous engineering as hardware designs. However, if we consider source code as design, we see that software designers actually do a considerable amount of validating and refining their designs. Software designers do not call it engineering; however, we call it testing and debugging.

Mock-ups, prototypes, and bread-boards are actually an accepted part of other engineering disciplines. Software designers do not have or use more formal methods of validating their designs because of the simple economics of the software build cycle.

Initially, the most common criticism can be summarized as "If source code is the design, then programmers are designers; but obviously they are not, therefore source code cannot be the design." Nobody states it that baldly, but when you parse what they do say, it comes down to the same thing. These are circular arguments that start with the assumption that programming/coding is a manufacturing type of activity. In logic, this is known as a "Begging the Question" fallacy. In essence, these people say, "your assumption (i.e., source code is the design) contradicts my assumption (i.e., programmers are assembly workers); therefore, your assumption must be wrong."

1.1 What Goes Wrong with Software?

If you are lucky, you start a project with a clear picture of what you want the system to be. The design of the system is a vital image in your mind. If you are luckier still, the clarity of that design makes it to the first release. Then something goes wrong. The software starts to rot like a piece of bad meat. As time goes by, the rot spreads and grows. Ugly festering sores and boils accumulate in the code, making it harder and harder to maintain.

Eventually, the sheer effort required to make even the simplest of changes becomes so onerous that the developers and front-line managers cry for a redesign. Such redesigns rarely succeed. Though the designers start out with good intentions, they find that they are shooting at a moving target. The old system continues to evolve and change, and the new design must keep up.

2 DESIGN SMELLS THE ODORS OF ROTTING SOFTWARE

The following are the symptoms of bad software designs, as defined:

- **Rigidity:**

Rigidity is the tendency for software to be difficult to change, even in simple ways. A design is rigid if a single change causes a cascade of subsequent changes in dependent modules. The more modules that must be changed, the more rigid the design.

Most developers have faced this situation in one way or another. They are asked to make what appears to be a simple change. They look the change over and make a reasonable estimate of the work required. But later, as they work through the change, they find that there are repercussions to the change that they hadnt anticipated. They find themselves chasing the change through huge portions of the code, modifying far more modules than they had first estimated. In the end, the changes take far longer than the initial estimate. When asked why their estimate was so poor, they repeat the traditional software developers lament, It was a lot more complicated than I thought!

- **Fragility:**

Fragility is the tendency of a program to break in many places when a single change is made. Often, the new problems are in areas that have no conceptual relationship with the area that was changed. Fixing those problems leads to even more problems, and the development team begins to resemble a dog chasing its tail.

As the fragility of a module increases, the likelihood that a change will introduce unexpected problems approaches certainty. This seems absurd, but such modules are not at all uncommon. These are the modules that are constantly in need of repair the ones that are never off the bug list, the ones that the developers know need to be redesigned.

- **Immobility(Reusability):**

A design is immobile when it contains parts that could be useful in other systems, but the effort and risk involved with separating those parts from the original system are too great. This is an unfortunate, but very common, occurrence.

- **Viscosity:**

Viscosity comes in two forms: viscosity of the software and viscosity of the environment. When faced with a change, developers usually find more than one way to make that change. Some of the ways preserve the design; others do not (i.e., they are hacks.)

When the design-preserving methods are harder to employ than the hacks, the viscosity of the design is high. It is easy to do the wrong thing, but hard to do the right thing. We want to design our software such that the changes that preserve the design are easy to make

Viscosity of environment comes about when the development environment is slow and inefficient. For example, if compile times are very long, developers will be tempted to make changes that don't force large recompiles, even though those changes don't preserve the design. If the source-code control system requires hours to check in just a few files, then developers will be tempted to make changes that require as few check-ins as possible, regardless of whether the design is preserved.

- **Needless Complexity:**

A design contains needless complexity when it contains elements that aren't currently useful. This frequently happens when developers anticipate changes to the requirements and put facilities in the software to deal with those potential changes. At first, this may seem like a good thing to do. After all, preparing for future changes should keep our code flexible and prevent nightmare changes later.

Unfortunately, the effect is often just the opposite. By preparing for too many contingencies, the design becomes littered with constructs that are never used. Some of those preparations may pay off, but many more do not. Meanwhile, the design carries the weight of these unused design elements. This makes the software complex and hard to understand.

- **Needless Repetition:**

Cut and paste may be useful text-editing operations, but they can be disastrous code-editing operations. All too often, software systems are built upon dozens or hundreds of repeated code elements.

When the same code appears over and over again, in slightly different forms, the developers are missing an abstraction. Finding all the repetition and eliminating it with an appropriate abstraction may not be high on their priority list, but it would go a long way toward making the system easier to understand and maintain.

- **Opacity(Understandability):**

Opacity is the tendency of a module to be difficult to understand. Code can be written in a clear and expressive manner, or it can be written in an opaque and convoluted manner. Code that evolves over time tends to become more and more opaque with

age. A constant effort to keep the code clear and expressive is required in order to keep opacity to a minimum.

When developers first write a module, the code may seem clear to them. That is because they have immersed themselves within it, and they understand it at an intimate level. Later, after the intimacy has worn off, they may return to that module and wonder how they could have written anything so awful. To prevent this, developers need to put themselves in their readers shoes and make a concerted effort to refactor their code so that their readers can understand it. They also need to have their code reviewed by others.

2.1 What Stimulates the Software to Rot?

In nonagile environments, designs degrade because requirements change in ways that the initial design did not anticipate. Often, these changes need to be made quickly, and they may be made by developers who are not familiar with the original design philosophy. So, though the change to the design works, it somehow violates the original design. Bit by bit, as the changes continue, these violations accumulate, and the design begins to smell.

We, as software developers, know full well that requirements change. Indeed, most of us realize that the requirements are the most volatile elements in the project. If our designs are failing due to the constant rain of changing requirements, it is our designs and practices that are at fault. We must somehow find a way to make our designs resilient to such changes and employ practices that protect them from rotting.

- VIOLATION OF DESIGN PRINCIPLES
- INAPPROPRIATE USE OF PATTERNS
- PROCEDURAL THINKING IN OO
- NONADHERENCE TO BEST PRACTICES AND PROCESSES

2.2 HOW TO ADDRESS SMELLS?

Clearly, smells significantly impact the design quality of a piece of software. It is therefore important to find, analyze, and address these design smells. Performing refactoring is the primary means of repaying technical debt.

2.3 DESIGN PRINCIPLES-BASED CLASSIFICATION OF SMELLS

While we were exploring possible classification schemes for design smells, we realized that it would be very useful to developers if they were aware of the design principle that was violated in the context of a smell. Thus, we sought to classify design smells as a violation of one or more design principles. Another factor that influenced our use of this classification scheme is that if we can easily trace the cause of smells to violated design principles, we can get a better idea of how to address them.

We treat the four major elements of Boochs object model as design principles and refer to them collectively as PHAME (Principles of Hierarchy, Abstraction, Modularization, and Encapsulation)

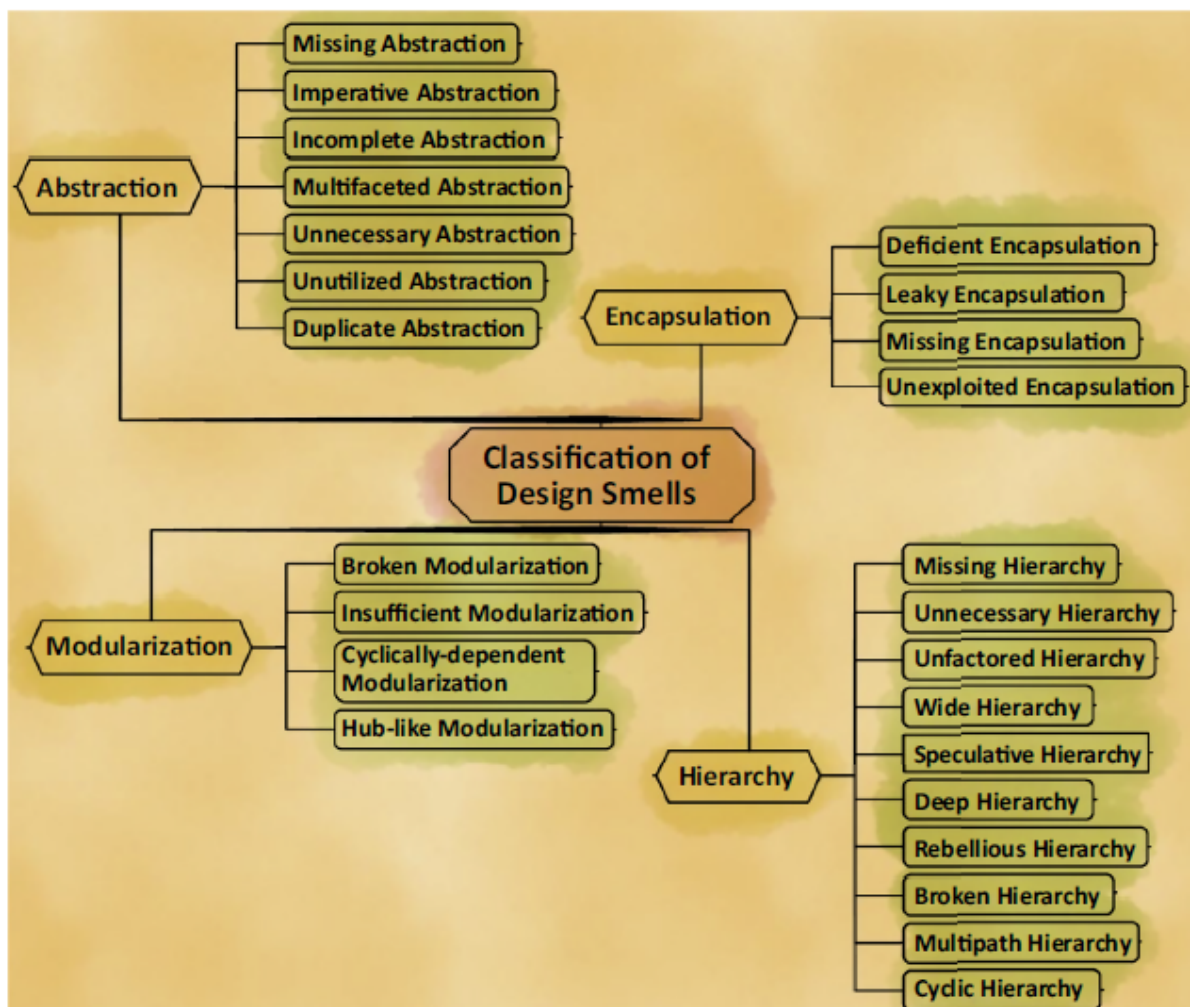


Figure 1: Design smells based on PHAME